

ASSIGNMENT-11.5

HT.N0-2203A52100

BATCH-50

TASK – 01:

Question:

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

Prompt: generate a docstring to generate a Stack class with push, pop, peek,

and is_empty methods. Sample Input Code: class Stack:

pass

Generated Code: class

Stack:

```
    """A      Last-  
InFirst-Out  
(LIFO)      data  
structure."""
```

```
def
```

```
    __init__(self):
```

```
        """Initialize
```

```
an empty
```

```
stack."""      self.items
```

```
=
```

```
[]
```

```
def push(self, item):  
    """Add an item to  
    the top of the  
    stack."""  
    self.items.append(item)
```

```
def pop(self):  
    """Remove and  
    return the top item from  
    the stack."""  
    if not  
self.is_empty():  
        return  
self.items.pop()  
    return None
```

```
def peek(self):  
    """Return the top  
    item without  
    removing  
    it."""  
    if not  
self.is_empty():  
        return self.items[-1]  
    return None
```

```
def is_empty(self):  
    """Check if the stack is  
    empty."""
```

```

        return len(self.items)

    ==

    0

    def
size(self):
    """Return the
    number of items
    in the stack."""
    return
    len(self.items)

```

Explanation:

This code implements a Stack data structure in Python using a list. It provides basic operations like push (add item), pop (remove top item), peek (view top item), and is_empty (check if stack is empty). The example shows how elements are added and removed following LIFO (Last In, First Out) order.

TASK – 02:

Question:

Task: Use AI to implement a Queue using Python lists.

Sample Input Code: class

Queue: pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt:

Write a Python program to implement a Queue using lists.

Start with this code: class

Queue:

pass

Implement FIFO operations:

enqueue() dequeue()

peek() size()

Generated Code: class

Queue:

```
    """A First-  
InFirstOut (FIFO) data  
structure."""
```

```
    def __init__(self):  
        """Initialize an  
empty queue."""  
        self.items = []
```

```
    def enqueue(self,  
item):    """Add an  
item to the rear of the  
queue."""  
    self.items.append(  
item)
```

```
    def dequeue(self):  
        """Remove and return the  
front item from the  
queue."""
```

```
        if not  
self.is_empty():
```

```

        return
self.items.pop(0)
return None

    def peek(self):
        """Return the front
        item without removing
        it."""
        if not
self.is_empty():
            return
self.items[0]
return None

    def is_empty(self):    """Check if the queue is empty."""
return len(self.items) == 0
        def
size(self):
        """Return the number of items
        in the queue."""
            return len(self.items)

```

Explanation:

This Python code creates a Queue class that follows the FIFO (First In, First Out) principle. It provides methods to add items (enqueue), remove items (dequeue), view the first item (peek), and check the size or if the queue is empty.

The example at the end shows how items are added to the queue and removed in the same order.

TASK – 03:

Question:

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
```

```
pass class LinkedList:
```

```
pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt:

generate a Singly Linked List with insert and display methods.

Sample Input Code: class Node:

```
pass class LinkedList:
```

```
pass
```

Expected Output:

- A working linked list implementation with clear method documentation.
generate a python program to implement a Singly Linked List with insert and display methods. **Generated Code:** class Node:

```
"""A node in a singly linked list."""
```

```
def __init__(self, data):
```

```
    """Initialize a node with data."""    self.data
```

```
= data    self.next = None class LinkedList:
```

```
    """A singly linked list implementation."""
```

```
def __init__(self):
```

```
    """Initialize an empty linked list."""    self.head
```

```
= None
```

```

def insert(self, data):
    """Insert a new node at the beginning of the list."""
    new_node = Node(data)

    new_node.next = self.head
    self.head = new_node

def insert_at_end(self, data):
    """Insert a new node at the end of the list."""
    new_node = Node(data)

    if not self.head:
        self.head = new_node
    return current = self.head
    while current.next:
        current = current.next
    current.next
    = new_node

def delete(self, data):
    """Delete the first node with the specified data."""
    if not self.head:
        return
    if self.head.data == data:
        self.head = self.head.next
        return
    current = self.head
    while current.next:
        if current.next.data == data:
            current.next = current.next.next
            return
        current = current.next

```

```

def display(self):
    """Print all elements in the linked list."""
    elements = []
    current = self.head
    while current:
        elements.append(str(current.data))
        current = current.next
    print(" → ".join(elements) if elements else
"Empty List")

def search(self, data):
    """Search for a node with the specified data."""
    current = self.head
    while current:
        if current.data == data:
            return True
        current = current.next
    return False

```

Explanation:

This code creates a singly linked list using two classes: Node and LinkedList. The Node class stores data and a reference to the next node, while the LinkedList class allows you to insert new nodes at the end and display all elements in order. The example inserts 1, 2, 3 into the list and prints them as 1 2 3.

TASK – 04:

Question:

Task: Use AI to implement a hash table with basic insert, search, and delete methods. Sample Input Code: class HashTable:

```

    pass

```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Prompt:

Write a Python program to implement a Hash Table with the operations insert, search, and delete.

Use collision handling with chaining (linked lists).

Create a class named HashTable.

Each method should be well commented and easy to understand.

Include a small example showing how to insert, search, and delete elements.

Generated Code:

```
class HashTable:
    """A hash table with
    collision handling
    using chaining."""

    def __init__(self,
size=10):
        """Initialize a hash
table with specified
size."""
        self.size =
size
        self.table = [[]
for
_ in range(size)]

    def _hash(self, key):
        """Generate
hash value
for a key."""
        return
hash(key)
% self.size

    def insert(self, key,
value):
        """Insert a
keyvalue pair into the
```

```

hash table."""
index =
self._hash(key)
    for i, (k, v) in
enumerate(self.table[i
n dex]):        if k == key:
self.table[ind ex][i] =
(key, value)
        return
self.table[index].a ppend((key,
value))

    def search(self, key):
        """Search for a value
by key."""        index
= self._hash(key)
        for k, v in
self.table[index]:
if k == key:
return v

return None

    def delete(self, key):        """Delete a keyvalue pair from the
hash table."""        index = self._hash(key)        self.table[index]
= [(k, v) for k, v in self.table[index] if k != key]

    def display(self):
        """Display all
keyvalue pairs in the

```

```

hash table."""
for i, bucket in
enumerate(self.table):
print(f"Bucket
{i}: {bucket}")

```

Explanation:

This code implements a Hash Table using chaining to store key-value pairs. The Node class stores a key, value, and link to the next node, while the HashTable class allows you to insert, search, and delete data using a hash function. If two keys map to the same index, they are stored in a linked list (chain) at that position.

TASK – 05:

Question:

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph: pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

Prompt:

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code: class

Graph: pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections. generate a python program to implement a graph using an adjacency list with methods to add vertices, add edges, and display connections.

Generated Code: class Graph:

```

"""A      graph
represented using an
adjacency list."""

```

```

def __init__(self):
    """Initialize an empty
graph."""
    self.graph = {}

def add_vertex(self, vertex):
    """Add a vertex to the
graph."""
    if vertex not in self.graph:
        self.graph[vertex]
= []

def add_edge(self, u, v,
weight=1):
    """Add an edge
between two
vertices."""
    if u not in self.graph:
        self.add_vertex(u)
    if v not in self.graph:
        self.add_vertex(v)
    self.graph[u].append((v,
weight))

def display(self):
    """Display the graph
connections."""
    for vertex, edges in
self.graph.items():
        print(f'{vertex}
: {edges}')

```

```

    def get_neighbors(self,
vertex):

        """Get all
neighbors of a
vertex."""

        return
self.graph.get(vertex,
[])

```

Explanation:

This code implements a Graph using an adjacency list in Python.

The class allows you to add vertices, create edges between vertices, and display the connections between them.

In the example, vertices A, B, C, and D are connected, and the display() method prints their relationships.

TASK – 06:

Question:

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack

Queue

Priority

Queue

Linked

List

Binary

Search

Tree

(BST) o

Graph o

Hash

Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt: generate python program for A hospital wants to build a Smart Hospital

Management System with the following features: 1. Patient Check-In System

2. Emergency Case Handling

3. Medical Records Storage

4. Doctor Appointment Scheduling

5. Hospital Room Navigation

Choose the most suitable data structure for each feature from this list:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

1. Create a table showing: Feature → Chosen Data Structure → Justification.
2. Write 2–3 simple sentences explaining why the data structure is suitable.
3. Implement one feature as a Python program with clear comments and docstrings.

Keep the explanation and code simple and beginner-friendly.

Generated Code:

```
class PatientCheckIn:

    """Patient check-in
    system using Queue -
    TASK 6
    Implementation."""

    def __init__(self):

        """Initialize the
        patient check-in
        queue."""

        self.queue =
        Queue()

        self.patient_id = 0

    def register_patient(self,
name, age):

        """Register a new patient."""

        self.patient_id +=
1

        patient = {"id":
self.patient_id, "name":
name, "age": age, "status":
"Waiting"}

        self.queue.enqueue
(patient)

        print(f"Patient
{name} registered with
ID {self.patient_id}")

    def call_next_patient(self):

        """Call the next
        patient for
        treatment."""
```

```

        if not self.queue.is_empty():

            patient = self.queue.dequeue()          patient["status"]
            = "Being Treated"

            print(f'Calling Patient:
{patient['name']} (ID:
{patient['id']})')
            return patient

            print("No patients
in queue")          return
            None

def display_queue(self):

    """Display all waiting
patients."""

    print(f'Patients in queue:
{self.queue.size()}')

```

Explanation:

Task 1: This code creates a table (list of tuples) that matches hospital system features with the most suitable data structures such as Queue, Priority Queue, Hash Table, Linked List, and Graph.

Task 2: It provides simple explanations of why each data structure is appropriate for those hospital features.

Task 3: It implements a Patient Check-In System using a Queue, where patients are added to the queue when they arrive and processed in FIFO (First-In-First-Out) order.

TASK – 07:

Question:

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.

5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue

o Linked List o

Binary Search Tree (BST) o

Graph o Hash Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

A city wants to build a Smart Traffic Management System with these features:

Traffic Signal Queue

Emergency Vehicle Priority Handling

Vehicle Registration Lookup

Road Network Mapping

Parking Slot Availability

Choose the most suitable data structure for each feature from this list:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

Create a table showing Feature → Data Structure → Justification.

Write 2–3 simple sentences explaining why the data structure is suitable.

Implement one feature as a Python program with comments and docstrings.

Keep the explanation and code very simple and easy to understand.

Generated Code:

```
class
TrafficSignalQueue:

    """Traffic signal
queue system - TASK
7 Implementation."""
    def __init__(self,
signal_id):

        """Initialize traffic signal
with queue."""

        self.signal_id = signal_id

        self.queue = Queue()

        self.vehicle_count
= 0

    def add_vehicle(self, vehicle_number):

        """Add vehicle to traffic
signal queue."""

        self.vehicle_count +=
1

        self.queue.enqueue
e({"id": self.vehicle_count,
"number":
vehicle_number})

        print(f'Vehicle
{vehicle_number} added to signal
{self.signal_id}')

    def
allow_passage(self):

        """Allow next vehicle
to pass."""

        if not self.queue.is_empty():
```

```

        vehicle =
self.queue.dequeue()

        print(f'Vehicle
{vehicle['number']}

can pass signal
{self.signal_id}")    return
vehicle

        print("No vehicles
waiting")    return
None

    def
get_waiting_count(self
):

        """Get number of vehicles
waiting."""

        return self.queue.size()

```

Explanation:

Task 1: The code creates a table (list of tuples) that links traffic management features with suitable data structures like Queue, Priority Queue, Hash Table, Graph, and Linked List.

Task 2: It explains why each data structure is appropriate for those traffic system features.

Task 3: It implements a Traffic Signal Queue using a Queue, where vehicles join the queue when they arrive and are processed in FIFO (First-In-First-Out) order at the signal.

TASK – 08:

Question:

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID. 5.

Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue o Priority Queue

o Linked List o

Binary Search Tree (BST) o

Graph o Hash Table o

Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Prompt:

An e-commerce company wants to build a Smart Online Shopping System with these features:

Shopping Cart Management

Order Processing System

Top-Selling Products Tracker

Product Search Engine

Delivery Route Planning

Choose the best data structure for each feature from this list:

Stack, Queue, Priority Queue, Linked List, Binary Search Tree (BST), Graph, Hash Table, Deque.

Tasks:

Create a table showing Feature → Data Structure → Justification.

Write 2–3 simple sentences explaining the choice.

Implement one feature as a Python program with comments and docstrings.

Keep the explanation and code simple and beginner-friendly. generate a python program to implement the Shopping Cart Management feature using a Stack data structure."""

Generated Code: class ShoppingCart:

```
    """Shopping cart using Linked List - TASK 8 Implementation."""

    def __init__(self):
        """Initialize empty shopping cart."""
self.cart = LinkedList()
self.total_price = 0

    def add_product(self, product_name, price):
        """Add product to cart."""
        self.cart.insert_at_end({"name":
product_name, "price": price})
        self.total_price += price
        print(f"Added {product_name}
({price}) to cart")

    def remove_product(self, product_name):
        """Remove product from cart."""
        self.cart.delete(product_name)
        print(f"Removed
{product_name} from cart")

    def display_cart(self):
        """Display
cart contents."""
        print("Shopping
Cart Contents:")
        self.cart.display()
        print(f"Total:
${self.total_price}")
```

Explanation:

Task 1: The code creates a table (list of tuples) that links e-commerce system features with suitable data structures such as Stack, Queue, Hash Table, Binary Search Tree, and Graph.

Task 2: It provides short explanations of why each data structure fits those features.

Task 3: It implements Shopping Cart Management using a Stack, where items are added and removed using the LIFO (Last-In-First-Out) principle, meaning the most recently added item is removed first.

OUTPUT:

```
PS C:\Users\krish\OneDrive\Documents\AI\Py.files> & c:\Users\krish\OneDrive\Documents\AI\Py.files\.venv\Scripts\python.exe "c:/Users/krish/OneDrive/Documents/AI/Py.files/ass-11.5 .py"
=====
DATA STRUCTURES DEMONSTRATION
=====

--- Hospital Patient Check-In ---
Patient John Doe registered with ID 1
Patient Jane Smith registered with ID 2
Calling Patient: John Doe (ID: 1)
Patients in queue: 1

--- Traffic Signal Queue ---
Vehicle ABC123 added to signal 1
Vehicle XYZ789 added to signal 1
Vehicle ABC123 can pass signal 1
Vehicles waiting: 1

--- Shopping Cart ---
Added Laptop ($999) to cart
Added Mouse ($25) to cart
Shopping Cart Contents:
{'name': 'Laptop', 'price': 999} -> {'name': 'Mouse', 'price': 25}
Total: $1024
PS C:\Users\krish\OneDrive\Documents\AI\Py.files> & c:\Users\krish\OneDrive\Documents\AI\Py.files\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\krish\OneDrive\Documents\AI\Py.files>
```